Week -1 Task -1:

Python Control Flow: Conditional Statements and Loops

Name: Kavya Biradar
Program: CodeMore, Python Programming Internship, Week 1, Task 1
Date: 18 October 2026
GitHub repository: https://github.com/kavya-n-biradar/python-control-flow

1. Introduction

This project uses conditional statements (if, elif, else) and loops (for, while) to control the flow of a Python program. I wrote two scripts. The first classifies a number as even or odd and as positive, negative or zero. The second uses loops to print a multiplication table, add up the first N natural numbers, and draw a right-angled triangle of asterisks. I tested both scripts with normal values and edge cases such as zero, negative numbers and invalid input.

2. Task 1: Number Classification (src/number_checks.py)

2.1 What the program does

The program asks the user for an integer and prints two results: whether the number is even or odd, and whether it is positive, negative or zero.

2.2 Logic

Reading the input. input() always returns text, so I convert it with int(). This line is inside a try block. If the user types something that is not a whole number, such as abc, Python raises a ValueError. The except ValueError block catches it and prints “Please enter a valid integer.” instead of crashing. The checks are placed in the else part of the try statement, so they run only when the input was valid.

Even or odd. The % (modulus) operator gives the remainder of a division. A number divided by 2 leaves remainder 0 if it is even and remainder 1 if it is odd. So number % 2 == 0 is true for even numbers, and the else branch handles odd numbers. This also works for negative numbers: -8 is even and -5 is odd.

Positive, negative or zero. There are three possible outcomes, so I used if / elif / else:

- number > 0 prints positive.
- number < 0 prints negative.

- otherwise the number must be zero.

Python checks the conditions from top to bottom and runs only the first one that is true. Since zero is neither greater nor less than zero, it falls through to else. Zero is also treated as even, because 0 % 2 is 0.

2.3 Code

```
# Program to classify an integer as even or odd, and as positive, negative or zero

# Ask the user to enter an integer and store it in a variable

try:

  number = int(input("Enter an integer: "))

except ValueError:

  print("Please enter a valid integer.")

else:

  # % modulus operator returns the remainder of a division operation.

  # A remainder of 0 when dividing by 2 means the number is even.

  if number % 2 == 0:

    print("The number is even.")

  else:

    print("The number is odd.")


  # Check if the number is positive, negative, or zero

  if number > 0:

    print("The number is positive.")

  elif number < 0:

    print("The number is negative.")

  else:

    print("The number is zero.")
```

2.4 Test results

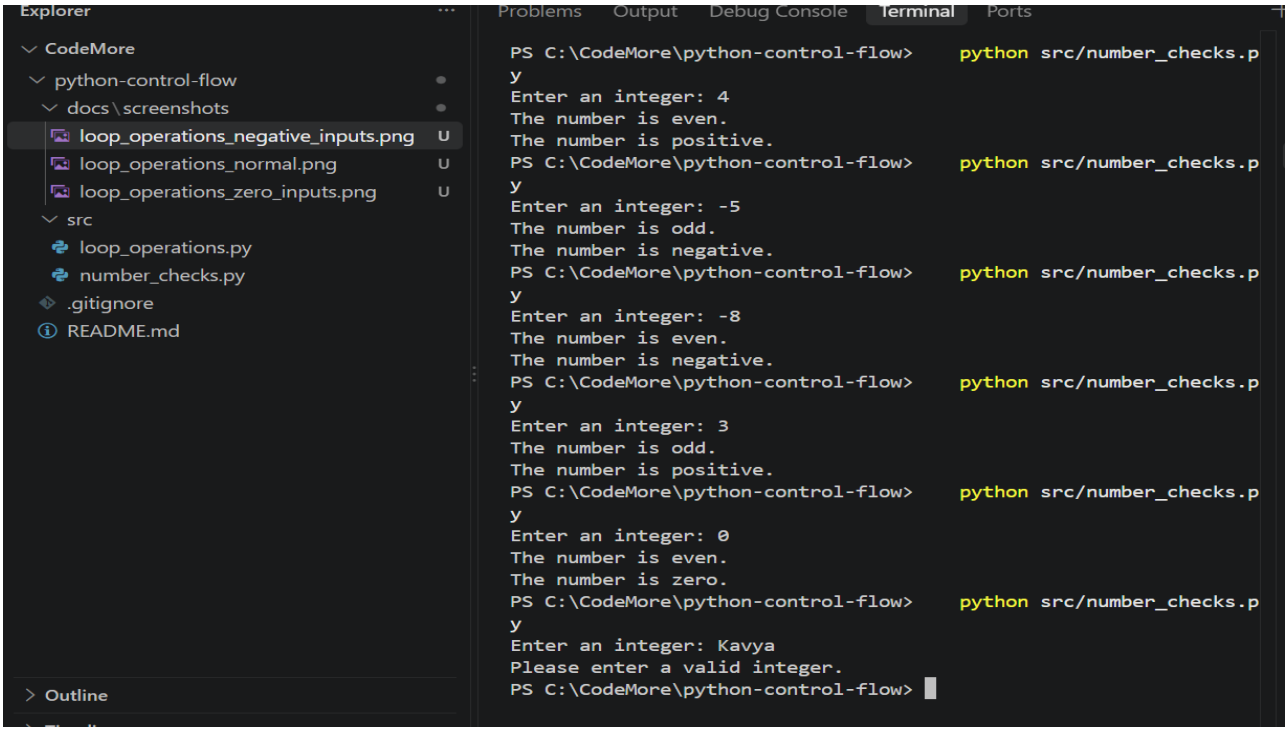


Fig 1: Output of Number Checks All Cases

3. Task 2: Loop Operations (src/loop_operations.py)

3.1 Multiplication table (for loop)

The program asks for a number and prints its table from 1 to 10. I used for i in range(1, 11). The range() function stops one before its second value, so to include 10 the end value has to be 11. In each pass, the loop variable i takes the next value and the program prints the line using an f-string, for example 5 x 3 = 15. A for loop suits this because I know the loop must run exactly 10 times.

Code:-

```
# This program prints the multiplication table of a given number.

num = int(input("Enter a number: "))  # Ask the user to enter a number

for i in range(1, 11):  # Loop through numbers 1 to 10
    print(f"{num} x {i} = {num * i}")  # Print the multiplication result
```

3.2 Sum of the first N natural numbers (while loop)

Natural numbers are 1, 2, 3 and so on. The program asks for N and adds the numbers from 1 to N using a while loop. It uses two variables: total, which starts at 0 and stores the running sum, and count, which starts at 1 and holds the number currently being added. While count <= n, the loop adds count to total and then increases count by 1.

The line count = count + 1 is essential. Without it, count would never exceed n, the condition would always stay true, and the loop would run forever.

Code:-

```
# This part finds the sum of the first N natural numbers using a while loop.

n = int(input("Enter a value for N: "))


if n < 1:
   # Natural numbers start from 1, so zero or negative N has no numbers to add
   print("N must be 1 or more, so the sum is 0.")
else:
   total = 0    # running total, starts at 0
   count = 1    # the number being added, starts at 1


   while count <= n:
      total = total + count   # add the current number to the total
      count = count + 1       # move to the next number
 print(f"The sum of the first {n} natural numbers is {total}.")
```

Example trace for N = 3:

Step	Count	Total After Adding	Condition for Next Pass
1	1	1	2 <= 3 is true
2	2	3	3 <= 3 is true
3	3	6	4 <= 3 is false, loop stops

The final result is 6, which matches 1 + 2 + 3.

Edge case: if N is 0 or negative, there are no natural numbers to add. An if n < 1 check prints “N must be 1 or more, so the sum is 0.” instead of running the loop.

3.3 Right-angled triangle of asterisks (for loop)

The program asks for the number of rows and prints a triangle where row i has i stars. For 4 rows the output is:

*
**
***
****

The loop is for i in range(1, rows + 1). As in the multiplication table, the + 1 makes sure the last row is included. In each pass the program prints "*" * i. In Python, multiplying a string by a number repeats it, so "*" * 3 gives ***. If the number of rows is less than 1, the program prints a message instead of drawing nothing.

3.4 Test results and screenshots

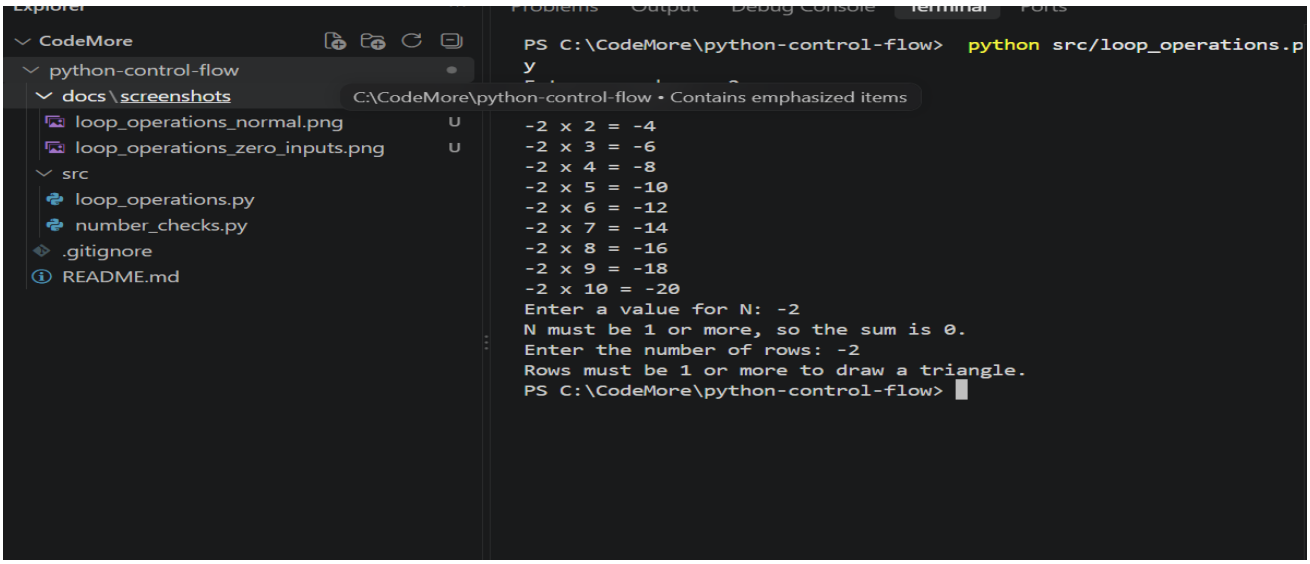


Fig 2: Output of Loop Operations Negative Inputs

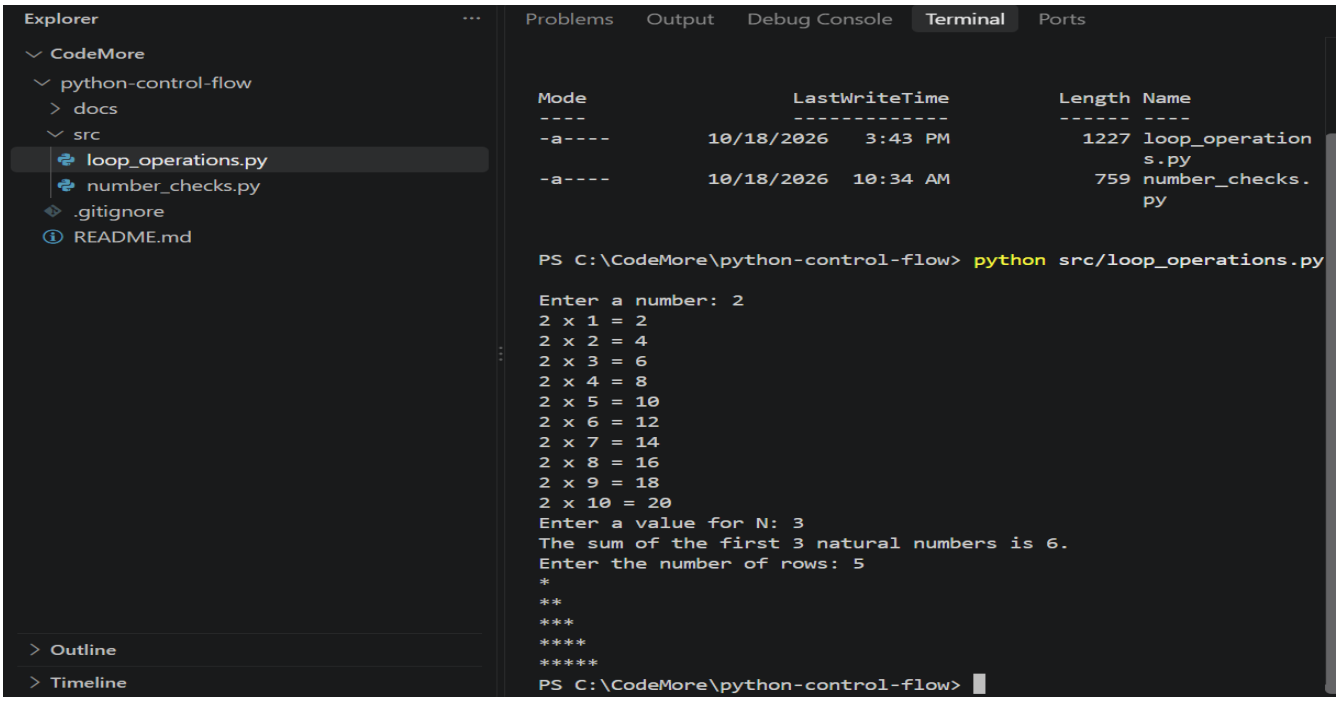


Fig 3: Loop Operations Normal

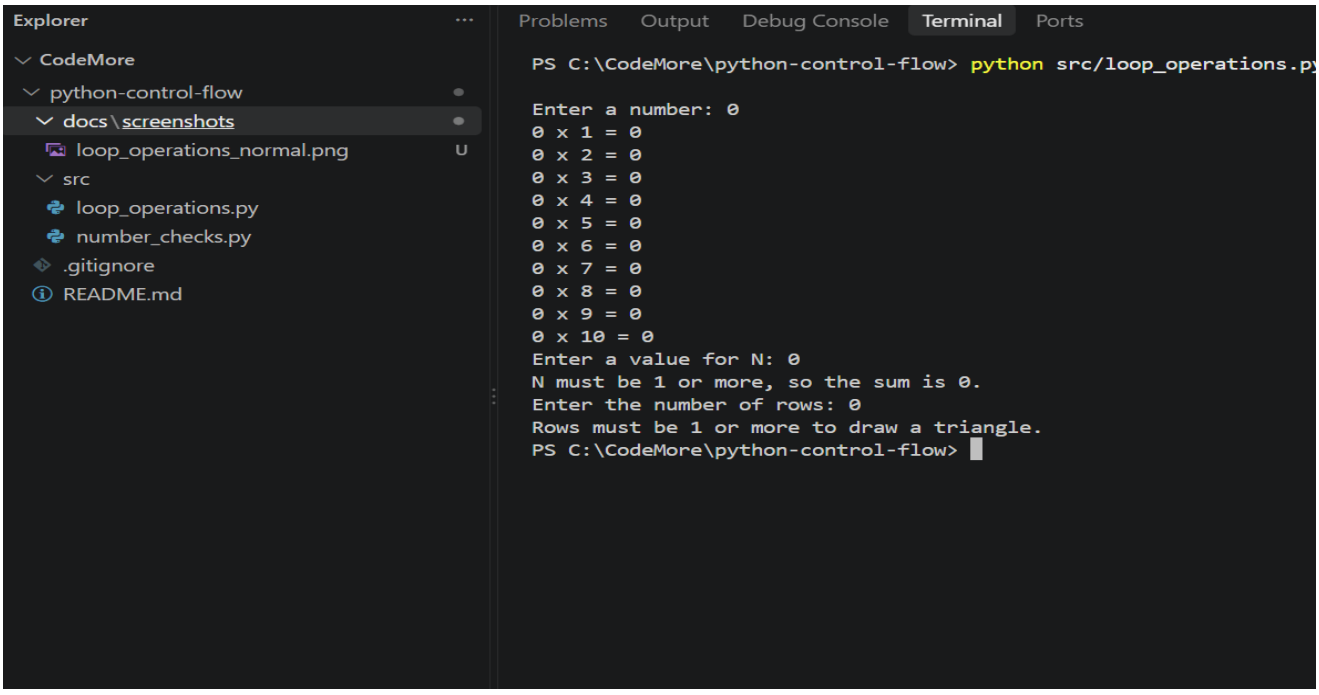


Fig 4: Loop Operations Zero Inputs

Figure 3 shows all three parts working with ordinary values. Figures 2 and 4 show that the program handles edge cases without crashing: the table of 0 gives only zeros, the table of a

negative number gives negative results, and the sum and triangle sections print explanatory messages.

4. Difference Between for and while Loops

Feature	for Loop	while Loop
Purpose	Repeats for each value in a sequence or range.	Repeats as long as a condition is true.
Best when	The number of repetitions is known in advance.	The number of repetitions is not fixed, or depends on a condition.
Counter	Handled automatically, for example by range().	Must be created and updated manually.
Risk	Very low.	An infinite loop if the condition never becomes false.
Used in this project	Multiplication table, triangle.	Sum of the first N natural numbers.

I used for for the table and the triangle because both repeat a fixed number of times (10 times, and rows times). I used while for the sum to practise a condition-controlled loop and to manage the counter myself. The sum could also be written with a for loop, but writing it with while made the role of the counter and the loop condition clearer.

5. Edge Cases Tested

- Zero: the number checker reports zero as even and zero. The table of 0 shows all results as 0. N = 0 and rows = 0 print messages.
- Negative numbers: % still correctly decides odd or even. The table shows negative results. Negative N and rows print messages.
- Invalid input: non-numeric input to number_checks.py is caught by try / except.

6. Challenges and Learnings

- Git and project setup: I learned to clone a repository, work in the correct folder, commit in small steps and push to GitHub. At first I ran commands from the wrong folder and had duplicate folders, and I learned to check my location in the terminal with pwd and dir.

- range() end value: I learned that range() stops before its second value, which is why the loops use 11 and rows + 1.
- Indentation: Python uses indentation to decide which lines belong to a block, and getting it right inside try / except / else took care.
- Edge cases: testing zero and negatives showed me that a program needs explicit handling for inputs it was not designed around.
- Comments: writing comments that explain why the code works, not only what it does, helped me understand my own logic.

7. Conclusion

This task helped me understand how conditions and loops control program flow. Using if / elif / else, for and while, I wrote two scripts that give correct results for normal input and handle edge cases cleanly. I also practised organising a project with a clear folder structure, a README, a .gitignore, documentation and a step-by-step commit history on GitHub.